데이터베이스 최적화 해보기 (1)

MySQL

박연종

2026.02.23.

System Software Lab.

실험을 위한 데이터베이스 생성

NYC Taxi and Limousine Commission에서 제공하는 2025년 Yellow Taxi Trip Records 사용 (1월 ~ 11월 내역)

약 78,039,389개 레코드가 존재하며, 용량은 약 8.6GB

예시 데이터

데이터 제공자 (제공자 ID)	미터기 시작 시각	미터기 종료 시각	승객 수	여행 거리 (mile)	미터기 시작 구역 (구역 ID)	미터기 종료 구역 (구역 ID)	요금 코드 (코드 ID)	저장 및 전송 플래그	결제 유형 (유형 ID)	금액 ($)	기타 요금 ($)	교통국 세금 ($)	개선 운임 ($)	팁 금액 ($)	톨게이트 금액 ($)	총 금액 ($)	미터기 시작 연도	미터기 시작 달
2	2088년 1월 24일 오전 12:25:39	2088년 1월 24일 오전 7:28:25	1	4.05	24	162	1	N	2	14.5	0	0.5	0.3	0	0	15.3	2088	1
2	2088년 1월 24일 오전 12:15:42	2088년 1월 24일 오전 12:19:46	1	0.63	41	166	1	N	2	4.5.	0	0.5	0.3	0	0	5.3	2088	1
2	2084년 11월 4일 오후 12:32:24	2084년 11월 4일 오후 12:47:41	1	1.34	238	236	1	N	2	10	0	0.5	0.3	0	0	10.8	2084	11
2	2084년 11월 4일 오후 12:25:53	2084년 11월 4일 오후 12:29:00	1	0.32	238	238	1	N	2	4	0	0.5	0.3	0	0	4.8	2084	11

문제 상황 (1)

미터기 시작 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

(결제 유형 중 직불카드의 ID는 1)

```
SELECT *
  FROM yellow_trip
 WHERE tpep_pickup_datetime
       BETWEEN '2025-03-28 00:00:00'
           AND '2025-03-31 23:59:59'
   AND payment_type = 1
 ORDER BY tpep_dropoff_datetime ASC
 LIMIT 10;
```

문제 상황 (1)

미터기 시작 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_dist
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1	

Grid | Text | Record

Refresh | Save | Cancel | Export data | 200 | 10 | 10 row(s) fetched - 1m 20s (0.005s fetch)

쿼리 검토

```
EXPLAIN ANALYZE SELECT * FROM …
```

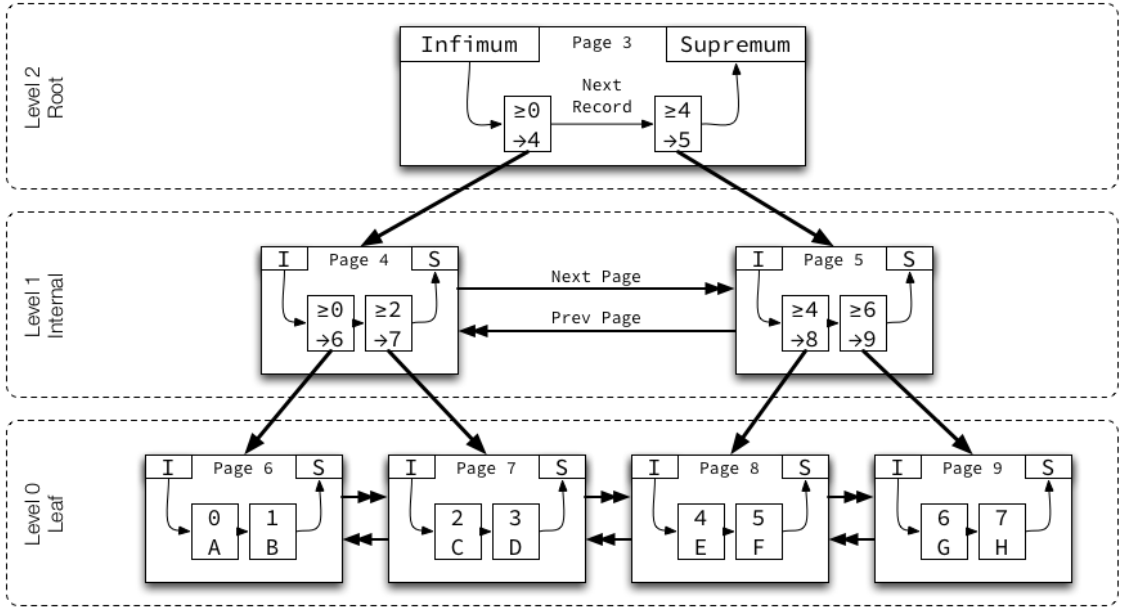
```
-> Limit: 10 row(s)  (cost=8.05e+6 rows=10) (actual time=71200..71200 rows=10 loops=1)
```

```
  -> Sort: yellow_trip.tpep_dropoff_datetime, limit input to 10 row(s) per chunk  (cost=8.05e+6
rows=75.7e+6) (actual time=71200..71200 rows=10 loops=1)
```

```
    -> Filter: ((yellow_trip.payment_type = 1) and (yellow_trip.tpep_pickup_datetime between
'2025-03-28 00:00:00' and '2025-03-31 23:59:59'))  (cost=8.05e+6 rows=75.7e+6) (actual
time=33612..71157 rows=338431 loops=1)
```

```
      -> Table scan on yellow_trip  (cost=8.05e+6 rows=75.7e+6) (actual time=13.1..63919
rows=77.6e+6 loops=1)
```

INDEX



InnoDB가 채택한 인덱스 구조, B+Tree

- 책의 색인(Index)처럼 레코드를 빠르게 찾기 위한 자료구조
- 항상 정렬된 상태를 유지

  CUD 작업이 빈번한 경우, 성능이 저하될 수 있음

인덱스 추가

```
-> Limit: 10 row(s)  (cost=8.05e+6 rows=10) (actual time=71200..71200 rows=10 loops=1)

  -> Sort: yellow_trip.tpep_dropoff_datetime, limit input to 10 row(s) per chunk  (cost=8.05e+6
rows=75.7e+6) (actual time=71200..71200 rows=10 loops=1)

    -> Filter: ((yellow_trip.payment_type = 1) and (yellow_trip.tpep_pickup_datetime between
'2025-03-28 00:00:00' and '2025-03-31 23:59:59'))  (cost=8.05e+6 rows=75.7e+6) (actual
time=33612..71157 rows=338431 loops=1)

      -> Table scan on yellow_trip  (cost=8.05e+6 rows=75.7e+6) (actual time=13.1..63919
rows=77.6e+6 loops=1)
```

미터기 시작 시각(tpep_pickup_datetime)에 인덱스를 추가하면

조회 속도가 빨라지지 않을까?

인덱스 추가

```
CREATE INDEX idx_tpep_pickup_datetime ON yellow_trip(tpep_pickup_datetime);
```

Execute time	2m 16s
Query	CREATE INDEX idx_tpep_pickup_datetime ON yellow_trip(tpep_pickup_datetime)

성능 개선 확인

인덱스 추가 전

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_dist
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1	

Refresh ▾ Save ▾ Cancel Export data ▾ 200 10 10 row(s) fetched - 1m 20s (0.005s fetch)

인덱스 추가 후

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1

Refresh ▾ Save ▾ Cancel Export data ▾ 200 10 10 row(s) fetched - 0.439s

성능 개선 확인

```
->Limit: 10 row(s)  (cost=1.37e+6 rows=10) (actual time=456..456 rows=10 loops=1)

  -> Sort: yellow_trip.tpep_dropoff_datetime, limit input to 10 row(s) per chunk  (cost=1.37e+6
rows=1.15e+6) (actual time=456..456 rows=10 loops=1)

    -> Filter: (yellow_trip.payment_type = 1)  (cost=1.37e+6 rows=1.15e+6) (actual time=2.12..425
rows=338431 loops=1)

      -> Index range scan on yellow_trip using idx_tpep_pickup_datetime over ('2025-03-28
00:00:00.000000' <= tpep_pickup_datetime <= '2025-03-31 23:59:59.000000'), with index condition:
(yellow_trip.tpep_pickup_datetime between '2025-03-28 00:00:00' and '2025-03-31 23:59:59')
(cost=1.37e+6 rows=1.15e+6) (actual time=2.11..413 rows=551459 loops=1)
```

문제 상황 (2)

미터기 종료 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

```
SELECT *
  FROM yellow_trip
 WHERE tpep_dropoff_datetime
       BETWEEN '2025-03-28 00:00:00'
           AND '2025-03-31 23:59:59'
   AND payment_type = 1
 ORDER BY tpep_dropoff_datetime ASC
 LIMIT 10;
```

문제 상황 (2)

미터기 종료 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

SELECT * FROM yellow_trip WHERE tpep_dropoff_date | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_dist
1	38,991,516	1	2025-03-27 23:49:43	2025-03-28 00:00:00	1	
2	38,993,109	2	2025-03-27 23:49:40	2025-03-28 00:00:00	1	
3	38,897,730	2	2025-03-27 09:49:48	2025-03-28 00:00:00	2	
4	38,982,391	2	2025-03-27 21:55:30	2025-03-28 00:00:00	1	
5	38,977,423	2	2025-03-27 21:43:33	2025-03-28 00:00:00	2	
6	38,911,090	2	2025-03-27 11:32:41	2025-03-28 00:00:00	5	
7	38,956,480	2	2025-03-27 18:50:11	2025-03-28 00:00:00	1	
8	38,930,306	2	2025-03-27 14:33:51	2025-03-28 00:00:00	1	
9	38,925,305	2	2025-03-27 14:33:40	2025-03-28 00:00:00	1	
10	38,934,831	2	2025-03-27 15:35:12	2025-03-28 00:00:00	6	

Refresh ▾ Save ▾ Cancel Export data ▾ 200 10 ··· 10 row(s) fetched - 1m 11s (0.001s fetch)

쿼리 검토

```
EXPLAIN ANALYZE SELECT * FROM …
```

```
-> Limit: 10 row(s)  (cost=8.06e+6 rows=10) (actual time=70792..70792 rows=10 loops=1)

  -> Sort: yellow_trip.tpep_dropoff_datetime, limit input to 10 row(s) per chunk  (cost=8.06e+6
rows=75.7e+6) (actual time=70792..70792 rows=10 loops=1)

    -> Filter: ((yellow_trip.payment_type = 1) and (yellow_trip.tpep_dropoff_datetime between
'2025-03-28 00:00:00' and '2025-03-31 23:59:59'))  (cost=8.06e+6 rows=75.7e+6) (actual
time=34672..70759 rows=338771 loops=1)

    -> Table scan on yellow_trip  (cost=8.06e+6 rows=75.7e+6) (actual time=0.865..63577
rows=77.6e+6 loops=1)
```

인덱스 추가

```
CREATE INDEX idx_tpep_dropoff_datetime ON yellow_trip(tpep_dropoff_datetime);
```

Execute time	2m 25s
Query	CREATE INDEX idx_tpep_dropoff_datetime ON yellow_trip(tpep_dropoff_datetime)

성능 개선 확인

SELECT * FROM yellow_trip WHERE tpep_dropoff_date | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,897,730	2	2025-03-27 09:49:48	2025-03-28 00:00:00	2
2	38,911,090	2	2025-03-27 11:32:41	2025-03-28 00:00:00	5
3	38,925,305	2	2025-03-27 14:33:40	2025-03-28 00:00:00	1
4	38,930,306	2	2025-03-27 14:33:51	2025-03-28 00:00:00	1
5	38,934,831	2	2025-03-27 15:35:12	2025-03-28 00:00:00	6
6	38,956,480	2	2025-03-27 18:50:11	2025-03-28 00:00:00	1
7	38,977,423	2	2025-03-27 21:43:33	2025-03-28 00:00:00	2
8	38,982,391	2	2025-03-27 21:55:30	2025-03-28 00:00:00	1
9	38,991,516	1	2025-03-27 23:49:43	2025-03-28 00:00:00	1
10	38,993,109	2	2025-03-27 23:49:40	2025-03-28 00:00:00	1

Grid Text Record

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.012s,

문제 상황 (3)

미터기 시작 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

```
SELECT *
  FROM yellow_trip
 WHERE tpep_pickup_datetime
       BETWEEN '2025-03-28 00:00:00'
           AND '2025-03-31 23:59:59'
   AND payment_type = 1
 ORDER BY tpep_dropoff_datetime ASC
 LIMIT 10;
```

문제 상황 (3)

미터기 시작 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh ▾ Save ▾ Cancel Export data ▾ 200 10 10 row(s) fetched - 29s,

문제 상황 (3)

미터기 시작 시간이 2025년 3월 28일 00:00:00부터 같은 해 3월 31일 23:59:59까지이고

결제 유형이 직불카드인 내역을 미터기 종료 시각이 빠른 순으로 10개 항목만 출력하세요.

미터기 시작 시각에만 인덱스가 있던 경우

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.439s

미터기 시작 시각과 종료 시각 모두에 인덱스가 있던 경우

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 29s,

쿼리 검토

```
EXPLAIN ANALYZE SELECT * FROM …
```

```
-> Limit: 10 row(s)  (cost=10.8 rows=0.999) (actual time=21084..21084 rows=10 loops=1)

  -> Filter: ((yellow_trip.payment_type = 1) and (yellow_trip.tpep_pickup_datetime between '2025-03-28 00:00:00' and '2025-03-31 23:59:59'))  (cost=10.8 rows=0.999) (actual time=21084..21084 rows=10 loops=1)

    -> Index scan on yellow_trip using idx_tpep_dropoff_datetime  (cost=10.8 rows=660) (actual time=0.81..20038 rows=10.6e+6 loops=1)
```

왜 다른 인덱스를 선택하는가?

For a query with an ORDER BY or GROUP BY and a LIMIT clause, the optimizer tries to choose an ordered index by default when it appears doing so would speed up query execution. In cases where using some other optimization might be faster, it is possible to turn off this optimization by setting the optimizer_switch system variable's prefer_ordering_index flag to off.

ORDER BY 또는 GROUP BY 절과 LIMIT 절이 포함된 쿼리의 경우, 옵티마이저는 쿼리 실행 속도가 향상될 것으로 판단되면 기본적으로 정렬된 인덱스를 선택하려고 시도합니다. 다른 최적화 방식이 더 빠를 수 있는 경우에는 optimizer_switch 시스템 변수의 prefer_ordering_index 플래그를 off로 설정하여 이 최적화를 비활성화할 수 있습니다.

- MySQL 8.4 Reference Manual의 10.2.1.19 LIMIT Query Optimization

MySQL 옵티마이저는 ORDER BY 또는 GROUP BY가 LIMIT와 함께 사용되면 ORDER BY에 지정된 속성의 인덱스를 기본 선택

idx_tpep_drop_datetime은 미터기 종료 시각을 오름차순으로 정렬한 인덱스

해당 순서대로 tpep_pickup_datetime이 WHERE의 범위에 있는지 검사 후 레코드를 선택

그래서 일치하는 레코드가 없으면 다음 레코드를 찾음

이때 메모리에 적재되어 있지 않은 레코드는 파일에서 불러와 검사 후 레코드 선택을 반복해 시간이 많이 소요

성능 개선 확인

prefer_ordering_index on

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 29s

prefer_ordering_index off

SELECT * FROM yellow_trip WHERE tpep_pickup_datet | Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 1.68s

어..? 실행 속도가 느려졌다

미터기 시작 시각에만 인덱스가 있던 경우

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.439s

prefer_ordering_index off

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 1.68s

미터기 종료 시각 인덱스 제거 후

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 1.645s (0.001s fetch)

재시작이 답이었다

미터기 시작 시각에만 인덱스가 있던 경우

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.439s

prefer_ordering_index off

SELECT * FROM yellow_trip FORCE INDEX (idx_tpep_p Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.739s,

미터기 종료 시각 인덱스 제거 후

SELECT * FROM yellow_trip WHERE tpep_pickup_date! Enter a SQL expression to filter results (use Ctrl+Space)

	trip_id	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip
1	38,997,905	2	2025-03-28 00:00:35	2025-03-28 00:00:41	1	
2	38,996,040	2	2025-03-28 00:00:45	2025-03-28 00:02:21	1	
3	38,996,988	1	2025-03-28 00:01:11	2025-03-28 00:02:35	2	
4	38,996,003	2	2025-03-28 00:00:38	2025-03-28 00:02:56	4	
5	38,996,349	2	2025-03-28 00:03:00	2025-03-28 00:03:34	1	
6	38,997,568	2	2025-03-28 00:00:03	2025-03-28 00:04:12	1	
7	38,996,834	2	2025-03-28 00:00:34	2025-03-28 00:04:49	1	
8	38,996,827	2	2025-03-28 00:00:33	2025-03-28 00:04:50	1	
9	38,998,438	1	2025-03-28 00:02:51	2025-03-28 00:05:16	1	
10	38,996,224	2	2025-03-28 00:00:48	2025-03-28 00:05:29	1	

Refresh Save Cancel Export data 200 10 10 row(s) fetched - 0.492s (0.001s fetch)

들어 주셔서 감사합니다

박연종

E-mail	park@duck.com
LinkedIn	yeonjong-park
Instagram	yeonjong.park
GitHub	patulus

2026.02.23.